

R2.02 – Intro aux IHM – JavaFX

HEFIED Mohamed Nail

Projet : Programmation d'un jeu de Morpion en JavaFX

(Durée : 2 séances de 3h soit 6h)

1 Contexte

Dans ce TP noté, vous allez concevoir et réaliser une application JavaFX d'un jeu très connu : le **Morpion**, ou **Tic-Tac-Toe**. Il s'agit d'un jeu pour 2 joueurs qui jouent chacun leur tour, avec pour objectif de former un alignement de symboles : O pour l'un des joueurs et X pour l'autre.

La version classique se déroule sur une grille de 3x3 cases, mais il existe de nombreuses variantes utilisant des grilles plus grandes, un plus grand nombre de joueurs, etc. Toutefois, afin de cadrer le travail attendu, les fonctionnalités à réaliser sont détaillées en section 2.2.

2 Travail à rendre

L'ensemble du travail devra être rendu sur la plateforme UMTICE, sous forme d'une archive .zip nommée : "R202_Nom_Prénom.zip". À la racine de cette archive, devront se trouver les éléments suivants :

1. Un dossier contenant le code source de l'application
2. Un rapport au format PDF nommé `Rapport_Nom_Prénom.pdf`

Pour cela :

1. Votre projet (File>New>Project...>JavaFX Project) devra être nommé **R202_Nom_Prénom** au moment de sa création.
2. À la fin de votre travail, exportez votre projet (Clic droit sur le projet dans le panneau Package Explorer>Export>General>Archive File>Assurez vous que tous vos fichiers sont sélectionnés>Browse >Nommez l'archive `R202_Nom_Prénom.zip`>Finish.
3. Glissez-déposez le rapport PDF dans le fichier `R202_Nom_Prénom.zip`

De plus, pour que je puisse importer et exécuter votre projet sans configuration supplémentaire, veuillez nommer votre User Library : "**JavaFX21**" en respectant la casse (majuscules et minuscules).

Si ces consignes de rendu ne sont pas respectées, une pénalité sera appliquée à la note finale (-1).

2.1 Le Rapport

Le rapport devra impérativement contenir (dans l'ordre suivant) :

- Une **explication succincte de la structure de votre application** : quels choix de séparation du code source vous avez faits, quels fichiers de ressources sont utilisés, etc.
- Indiquez simplement **ce que vous avez réussi à faire et ce qui n'a pas abouti**. Soyez honnête. Si vous avez commencé certaines fonctionnalités sans parvenir à les terminer, mentionnez-le et expliquez brièvement les difficultés rencontrées.
- Des **captures d'écran de toutes vos vues** : cela me permettra de voir le résultat que vous obtenez de votre côté dans l'éventualité où je n'arriverais pas à exécuter votre application.
- Un **manuel d'utilisation** est nécessaire à la bonne compréhension de votre application si celle-ci est complexe.

Vous êtes bien sûr libre d'ajouter des choses (du contenu, des images, des dessins, etc.), mais il faut au minimum cette base.

Le rapport comptera dans la note finale.

2.2 Liste des fonctionnalités du jeu

Certaines fonctionnalités sont **obligatoires** et doivent impérativement être implémentées. Cependant, même si toutes sont réalisées correctement, cela ne suffira pas pour obtenir la note maximale. D'autres fonctionnalités sont proposées à titre **optionnel** pour enrichir l'application : il n'est pas nécessaire de toutes les réaliser, mais il faudra tout de même en implémenter quelques-unes pour viser la note maximale. Enfin, les fonctionnalités optionnelles ne seront pas prises en compte si les fonctionnalités obligatoires ne sont pas réalisées.

Fonctionnalités obligatoires:

- Un titre sur chaque fenêtre.
- Plusieurs fenêtres (modales et non modales).
- Une barre de menu permettant **a minima** de réinitialiser le jeu, de voir les règles du jeu et de quitter le jeu.
- Des éléments permettant de nommer les deux joueurs.
- Une interface permettant de choisir qui joue en premier : « Joueur 1 », « Joueur 2 », ou choix aléatoire.
- Dans la partie principale de la fenêtre principale : la zone de jeu avec 9 cases (3x3). Le type d'élément utilisé pour les cases est libre (boutons, canvas, ou autre : à vous de voir).
- À chaque clic dans une case, celle-ci doit afficher le symbole du joueur dont c'est le tour : un O ou un X. Le changement de joueur doit s'effectuer automatiquement après chaque clic.
- À chaque tour de jeu, le joueur dont c'est le tour doit le savoir explicitement par une indication à l'écran (sous la forme de votre choix).

- La fin de jeu doit se faire automatiquement : lorsqu'un joueur a aligné trois symboles, le jeu doit s'arrêter et indiquer (sous la forme de votre choix) le nom du joueur vainqueur. En cas de match nul (c'est-à-dire si toutes les cases ont été remplies sans avoir trois symboles identiques alignés), l'application doit le mentionner.
- À la fin du jeu, une interface doit permettre de rejouer ou de quitter l'application. En choisissant « Rejouer », la grille de jeu doit se réinitialiser. Autre option à prendre en compte : garder les mêmes noms de joueurs ou changer.
- À la fin de chaque partie, établir un tableau de scores montrant le nombre de victoires de chacun des joueurs.

Fonctionnalités optionnelles :

- À la fin de la partie, afficher l'alignement gagnant à l'aide d'un effet de votre choix (surbrillance des cases, ligne barrée sur l'alignement, etc.).
- À la place des symboles textuels X et O, utiliser des images.
- Permettre aux joueurs de personnaliser leur symbole (textuel avec éditeur de police et de couleur, image, autre...).
- Implémenter un mode « Un joueur », permettant de jouer seul face à l'ordinateur :
 - Niveau 1 : l'ordinateur joue aléatoirement dans une case libre.
 - Niveau 2 : l'ordinateur choisit le meilleur coup pour tenter de gagner.
- Implémenter la possibilité de choisir la taille de la grille, pour jouer à des variantes à 16 cases, 25 cases, ou plus.
- Générer un fichier JAR de votre application et l'ajouter à votre archive. Ce format permet de publier des applications Java (par exemple à des utilisateurs finaux).

Conseils techniques:

- Vous pouvez utiliser FXML comme vous pouvez programmer en approche procédurale, ou mixer les deux. N'hésitez pas à **justifier vos choix dans votre rapport**.
- Vous pouvez coder les fonctionnalités dans l'ordre que vous voulez. Par exemple, si la fonctionnalité X vous pose problème, passez à la fonctionnalité Y et revenez à la X plus tard. Ceci vous permettra de perdre le moins de temps possible.
- Faites bien attention à la propreté de votre code.
- Pensez à commenter votre code pour faciliter la relecture.
- Essayez d'appliquer un minimum de style à votre application afin de la rendre sympa et de la différencier de celle de votre voisin.

3 Règles et consignes de travail

- Les différentes ressources utilisées jusqu'ici (cours, TD, documentation Java, etc.) sont autorisées.
- Il existe de nombreux exemples de morpion sur Internet. Vous pouvez vous en inspirer, **sans les copier aveuglément** : l'objectif est de vous amener à appliquer vous-même les notions vues en TD. **Votre rapport devra montrer que ce que vous avez mis en œuvre correspond à ce que vous avez compris et retenu.**
- L'IA générative est autorisée, mais **il est formellement interdit de lui demander de répondre aux questions ou de copier-coller le code généré.** En revanche, vous pouvez la solliciter pour des questions de conception, d'organisation du code ou d'algorithmique liées à la logique du jeu. **Toute utilisation de l'IA dans ce cadre devra être explicitement mentionnée dans le rapport** (par exemple : « ChatGPT m'a proposé cela, j'ai compris ceci et j'ai appliqué cela »). Gardez toutefois à l'esprit que ses réponses peuvent être erronées (hallucinations) : l'objectif reste d'appliquer ce que vous avez retenu, en privilégiant le cours et la documentation officielle pour la partie code.
- **Le plagiat se repère facilement et sera sanctionné sévèrement.**
- Votre **assiduité pendant les séances de TP sera prise en compte** : si je remarque que vous ne travaillez pas sur votre projet, des sanctions seront appliquées à la note.